```
# import libs
from scipy.io import loadmat
import numpy as np
from itertools import combinations
import numpy as np
from scipy.optimize import linprog
import time
```

```
data = loadmat("COMP5340HW1.mat")
print(data.keys())

# Prepare data
Af = data['Af']
Ar = data['Ar']
yf = data['yf'].ravel()
yr = data['yr'].ravel()

print("Af shape:", Af.shape)
print("Ar shape:", Ar.shape)
print("yf shape:", yf.shape)
print("yr shape:", yr.shape)
```

```
dict_keys(['__header__', '__version__', '__globals__', 'Af', 'Ar', 'yf', 'yr'])
Af shape: (25, 100)
Ar shape: (25, 100)
yf shape: (25,)
yr shape: (25,)
```

Problem:

$$y = Ax$$

- $A_f, A_r \in \mathbb{R}^{25\times 100}$: fat sensing matrices
- $y_f, y_r \in \mathbb{R}^{25}$: measurement vectors
- $x \in \mathbb{R}^{100}$: original signal

$$S \leq 3$$

Recover the sparse signal x from:

$$y_f = A_f x \quad \text{and} \quad y_r = A_r x$$

```
def l0_exhaustive_recovery(A, y, S=3, tol=1e-6):
    """
    Exhaustive search for sparse recovery.

    A: (M, N)
    y: (M,)
    """
    _, N = A.shape
    y = y.ravel()

    best_x = None
    best_error = float("inf")

    # Try sparsity levels 1 → S
    for s in range(S + 1):
        for support in combinations(range(N), s):
            support = list(support)

            # Submatrix with selected columns
            A_s = A[:, support]

            # Solve least squares: A_s * x_s ≈ y
            x_s, _, _, _ = np.linalg.lstsq(A_s, y, rcond=None)

            # Reconstruct full x
            x = np.zeros(N)
            x[support] = x_s

            # Compute residual
            error = np.linalg.norm(y - A @ x)
```

```
            # Check if exact (or best)
            if error < best_error:
                best_error = error
                best_x = x

            if error < tol:
                # print(f"Exact solution found with sparsity {s}")
                return x

    return best_x
```

```
# Recover signals
x_fl0 = l0_exhaustive_recovery(Af, yf)
x_rl0 = l0_exhaustive_recovery(Ar, yr)

print("Recovered x (Af):", x_fl0)
print("Recovered x (Ar):", x_rl0)
```

```
Recovered x (Af): [ 0.  0.  5.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  3.  0.  0.
  0.  0.  0.  0.  0.  0.  0. 40.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
Recovered x (Ar): [ 0.  0.  5.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  3.  0.  0.
  0.  0.  0.  0.  0.  0.  0. 40.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
```

Question 1

- **Exhaustive-search (ℓ_0-minimization):**
  Yes, the sparse signal x can be exactly recovered.

```
def l1_linear_programming(A, y):
    """
    Solve min ||x||_1 subject to Ax = y using linear programming.
    """
    _, N = A.shape
    y = y.ravel()

    # Variables: [u, v] ∈ R^{2N}
    c = np.ones(2 * N)  # minimize sum(u + v)

    # Equality constraint: A(u - v) = y
    A_eq = np.hstack([A, -A])
    b_eq = y

    # Bounds: u >= 0, v >= 0
    bounds = [(0, None)] * (2 * N)

    result = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')

    if not result.success:
        raise ValueError("LP did not converge")

    u = result.x[:N]
    v = result.x[N:]

    x = u - v
    return x
```

```
# Recover signals
x_fl1 = l1_linear_programming(Af, yf)
print("Recovered x (Af):", x_fl1)
```

```
Recovered x (Af): [ 0.00000000e+00  1.71587949e-14  5.00000000e+00  0.00000000e+00
 -5.55518686e-14  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  3.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
 -0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
 -0.00000000e+00  4.00000000e+01  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  5.69772448e-14 -0.00000000e+00
```

```
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
 -6.62690231e-14  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
 -7.46832811e-14  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00 -2.50113477e-13
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  4.09009399e-14  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
```

```
# Recover signals
x_fl1 = l1_linear_programming(Ar, yr)
print("Recovered x (Ar):", x_fl1)
```

```
Recovered x (Ar): [ 0.  0.  5.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  3.  0.  0.
  0.  0.  0.  0.  0.  0.  0. 40.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0. -0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0. -0.  0.  0.  0.  0.  0.]
```

```
# Exhaustive ℓ0
start = time.time()
x_fl0 = l0_exhaustive_recovery(Af, yf)
t_fl0 = time.time() - start
print(f"Exhaustive ℓ0 recovery (Af) took {t_fl0:.4f} seconds")

start = time.time()
x_rl0 = l0_exhaustive_recovery(Ar, yr)
t_rl0 = time.time() - start
print(f"Exhaustive ℓ0 recovery (Ar) took {t_rl0:.4f} seconds")

# LP ℓ1
start = time.time()
x_fl1 = l1_linear_programming(Af, yf)
t_fl1 = time.time() - start
print(f"LP ℓ1 recovery (Af) took {t_fl1:.4f} seconds")

start = time.time()
x_rl1 = l1_linear_programming(Ar, yr)
t_rl1 = time.time() - start
print(f"LP ℓ1 recovery (Ar) took {t_rl1:.4f} seconds")
```

```
Exhaustive ℓ0 recovery (Af) took 0.5120 seconds
Exhaustive ℓ0 recovery (Ar) took 0.5353 seconds
LP ℓ1 recovery (Af) took 0.0221 seconds
LP ℓ1 recovery (Ar) took 0.0215 seconds
```

Question 2

- **Linear programming (ℓ_1-minimization):**
  A function based on linear programming can recover x by solving $\min \|x\|_1$ subject to $Ax = y$. This convex relaxation successfully recovers the same sparse signal as ℓ_0 in this problem.
- **Comparison (results and runtime):**
  Both methods yield identical recovery results (same support and values). However, ℓ_0 is computationally expensive due to combinatorial complexity, while ℓ_1 is significantly faster and more scalable.

Question 3: Observations on Sensing Matrices and Recovered Signals

- **Underdetermined system**

  Both sensing matrices A_f and A_r have dimensions:

$$A \in \mathbb{R}^{25\times 100}, \quad M = 25 \ll N = 100$$

  This means the system $y = Ax$ is underdetermined, so infinitely many solutions exist. Sparse recovery is only possible due to the constraint $S \leq 3$.

- **Identical recovery results**

  Both sensing matrices recover the same sparse signal:

  - Support (nonzero indices): $[2, 15, 25]$
  - Values: $[5, 3, 40]$

  This indicates that:

  - The recovered signal is consistent across both matrices
  - The sparse solution is likely unique under the constraint $S \leq 3$

- **Conclusion**

  Both sensing matrices A_f and A_r are effective for sparse recovery despite being underdetermined. The results show that:

  - Both matrices preserve sufficient information about the sparse signal
  - The recovered solution is unique

  Overall, both A_f and A_r are suitable sensing matrices for sparse recovery at low sparsity levels.

```
np.random.seed(0)

# generate random sparse signal
def generate_sparse_x(N, S):
    x = np.zeros(N)
    support = np.random.choice(N, S, replace=False)
    values = np.random.randint(1, 50, size=S)
    x[support] = values
    return x, support

def run_experiment_exhausive(A, name):
    _, N = A.shape
    print(f"\nRunning exhaustive recovery for matrix: {name}")
    for S in range(3, 5):
        x_true, _ = generate_sparse_x(N, S)
        y = A @ x_true

        start = time.time()
        x_l0 = l0_exhaustive_recovery(A, y, S)
        t_l0 = time.time() - start

        # Errors
        err_l0 = np.linalg.norm(x_l0 - x_true)

        print(f"S = {S}")
        print(f" Exhaustive search error: {err_l0:.4e}, time: {t_l0:.2f}s")

def run_experiment_lp(A, name):
    _, N = A.shape
    print(f"\nRunning l1 recovery for matrix: {name}")
    for S in range(3, 11):
        x_true, _ = generate_sparse_x(N, S)
        y = A @ x_true

        start = time.time()
        x_l1 = l1_linear_programming(A, y)
        t_l1 = time.time() - start

        # Errors
        err_l1 = np.linalg.norm(x_l1 - x_true)

        print(f"S = {S}")
        print(f" Linear programming l1 error: {err_l1:.4e}, time: {t_l1:.2f}s")
```

```
run_experiment_lp(Af, "Af")
run_experiment_lp(Ar, "Ar")
```

```
Running l1 recovery for matrix: Af
S = 3
 Linear programming l1 error: 1.9518e-12, time: 0.03s
S = 4
 Linear programming l1 error: 1.4296e-12, time: 0.03s
S = 5
 Linear programming l1 error: 1.7891e-13, time: 0.03s
```

```
S = 6
 Linear programming l1 error: 9.3718e-13, time: 0.03s
S = 7
 Linear programming l1 error: 2.0389e-13, time: 0.03s
S = 8
 Linear programming l1 error: 4.9768e+01, time: 0.02s
S = 9
 Linear programming l1 error: 1.5495e+01, time: 0.03s
S = 10
 Linear programming l1 error: 9.5693e+01, time: 0.04s

Running l1 recovery for matrix: Ar
S = 3
 Linear programming l1 error: 1.3774e-12, time: 0.03s
S = 4
 Linear programming l1 error: 2.7576e-14, time: 0.04s
S = 5
 Linear programming l1 error: 1.9175e-13, time: 0.03s
S = 6
 Linear programming l1 error: 3.3835e-13, time: 0.03s
S = 7
 Linear programming l1 error: 4.8800e+01, time: 0.02s
S = 8
 Linear programming l1 error: 1.8985e+01, time: 0.03s
S = 9
 Linear programming l1 error: 5.0500e+00, time: 0.03s
S = 10
 Linear programming l1 error: 1.4318e+01, time: 0.03s
```

```
run_experiment_exhausive(Af, "Af")
run_experiment_exhausive(Ar, "Ar")
```

```
Running exhaustive recovery for matrix: Af
S = 3
 Exhaustive search error: 2.8643e-14, time: 1.20s
S = 4
 Exhaustive search error: 1.9132e-14, time: 76.41s

Running exhaustive recovery for matrix: Ar
S = 3
 Exhaustive search error: 2.1316e-14, time: 5.16s
S = 4
 Exhaustive search error: 1.5838e-14, time: 68.15s
```

Question 4: Observations with Increasing Sparsity

- **Low sparsity ($S = 3 \sim 5$):** Both ℓ_0 and ℓ_1 methods achieve exact recovery with near-zero error. The reconstruction is stable, and ℓ_1 is significantly faster.
- **Medium sparsity ($S = 6 \sim 7$):** ℓ_0 still recovers the signal accurately, but ℓ_1 may start to show slight errors. Recovery becomes less stable as sparsity increases.
- **High sparsity ($S = 8 \sim 10$):** ℓ_1 often fails to recover the correct signal, while ℓ_0 becomes computationally infeasible due to exponential complexity $\binom{100}{10} \approx 1.73 \times 10^{13}$. Exact recovery is no longer guaranteed.